

Speaker notes — Member 6 — ISSD

Main talk: slides 1–6. Backup evidence and references: slides 7–10. The live transition is **slide 4** → **LIVE_DEMO Part 1**; slide 5 uses recorded Task 2 evidence. Follow Part 0 before the slot and Part 3 afterward. The suggested 5–7 minutes is provisional, not a measured rehearsal or confirmed allowance.

Slide 1 — Private should stay private

Introduce Dirty COW, CVE-2016-5195, as a historical local kernel race. Copy-on-write should keep a process's private changes separate from the original file. This vulnerability can break that boundary and modify data an ordinary user cannot normally write. Member 5's example was in a deliberately privileged application; here the vulnerable component is the kernel. Explain this before the live demonstration. The fresh work took place on 9 October 2026 in the separate SEED12 guest. Suggested 40 seconds.

Slide 2 — Normal COW and the two workers

Trace the normal lane first: read-only-opened file, writable private mapping, private copy changes and backing file stays intact. Then trace the attack: target `O_RDONLY`, `PROT_READ` plus `MAP_PRIVATE`, one worker repeatedly writing its own `/proc/self/mem` and another repeatedly discarding mapping state with `MADV_DONTNEED`. The kernel's concurrent COW/page handling is the vulnerable boundary. `pwrite`'s offset is a virtual address in this process; the program did not gain ordinary file-write permission. The diagram is conceptual, not a measured kernel trace. Suggested 50 seconds.

Slide 3 — Verified environment and normal protection

State the actual environment: Ubuntu 12.04.2, i686/32-bit, kernel 3.5.0-37-generic, package 3.5.0-37.58~precise1 from linux-lts-quantal, seed UID 1000. The directory name `Labs_20.04` does not change the official requirement for the old image. The programs were built on guest ext4 and remained seed-owned 0755; no Set-UID installation was used. Point to S04: `root:root 0644`, original digits and Permission denied. Its normal file protection works. These are recorded observations, with the actual full environment in S02. Suggested 45 seconds.

Slide 4 — Switch to the live dummy

Use the already-prepared same Ubuntu terminal and `LIVE_COMMANDS`. Show `id`, permissions and the original file. Attempt the ordinary write, then run `cow_control` and reread the file/hash. Only after that baseline passes, run the supplied wrapper with the new label and 15-second limit. Describe the actual result and verify complete bytes/metadata. Do not promise a win. If this run times out or is partial, preserve it and show recorded S07 from 9 October, explicitly labelled recorded. The original fresh-t1-01 used a 30-second bound and measured 0 integer seconds; no finer time or kernel attempt count was measured. Return to slide 5. Suggested 90 seconds.

Slide 5 — Recorded UID change and actual root login

Say clearly: this is recorded Task 2 evidence from 9 October. Charlie began at UID 1001 and GID 1002. The sole fresh-t2-01 race, run as seed, changed only the four-character UID field to 0000; all other file bytes and mode/owner stayed the same. The first authentication failed for an unestablished reason. The retry was another non-`sudo su - charlie`, followed by actual `id`, `id -u = 0` and `whoami root`. The numeric UID is the privilege proof; GID remained 1002. The proof shell was exited before full restoration and a new UID-1001 login. Use backup slides 8/9 for the exact diff or restoration, or the labelled native-recording fallback if time allows. Suggested 50 seconds with the static proof; full video needs extra time.

Slide 6 — Fix the kernel and verify the final state

Cover all four allocation themes: patch the kernel and boot the fixed version; maintain a supported updated OS; limit unnecessary local accounts/untrusted code execution; monitor account integrity, unexpected numeric UID-0 entries and privileged sessions. Monitoring aids detection and does not repair COW. Standard target permissions were already restrictive. Member 5's symlink protection or removing unrelated Set-UID bits cannot fix this kernel bug. The optional patched comparison was not performed; discussion only. Our final full baseline and fresh UID-1001 login passed; no test shell/attacker or dummy remained, and evidence was exported before normal shutdown. Close with private changes must remain private. Suggested 60 seconds. Run LIVE_DEMO Part 3 after the live session.

Slide 7 — Recorded Task 1 fallback

Use this if the new live race does not succeed or the lecturer asks for complete backing-file verification. S07 is an actual 9 October framebuffer excerpt. It shows the full before/after diff, fresh cat, root:root 0644 and 19 bytes, complete expected-byte comparison returning 0 and the original elapsed reading. The attacker was already stopped. This recorded result does not change or replace the outcome of a later live attempt.

Slide 8 — Exact account change

Point to both complete charlie lines in the diff: 1001 became 0000; the GID remains 1002 and other fields match. The verifier compared all 2040 bytes, not just this displayed line. Metadata remained root:root 0644. The numeric UID-0 query found root and charlie after the attack. Explain why a broad search for a number or a one-byte overwrite would be unsuitable, and why the actual original UID was discovered rather than assumed. The two original zero positions stay zero; four is the field width, not the number of necessarily different bytes.

Slide 9 — Restoration includes a new login

This is S11, after proof shell PID 3392 had ended and been checked absent. The protected full-file comparison and original hash pass. A new non-sudo charlie login reports UID 1001, GID 1002 and whoami charlie, then exits to seed. Restoring a database does not revoke credentials already held by a process. S13 separately establishes no experiment/su/root shell, original root-only UID-0 list, absent dummy and ordinary binaries. Both transcripts and the final archive were closed and verified before shutdown.

Slide 10 — Sources and assistance

Attribute the task/exploit pattern to Wenliang Du / SEED Labs, Copyright 2017, CC BY-NC-SA 4.0. The report includes the official task/source, historical VM manual, Linux memory-interface manuals, vendor CVE information and lecturer allocation. The four supplied C/build/trial files were unchanged. GPT-6 Astra through OpenCode helped with public guest command delivery, genuine captures, verification and document preparation; the user supplied host settings views and entered passwords interactively. The folder name does not identify the model used. No oral delivery or upload has occurred automatically.

Lecturer Q&A

Question	Concise answer
Why Ubuntu 12.04 when the link says 20.04?	The official lab specifies the historical 32-bit SEED12 VM; the URL category is not the runtime requirement.
Is every 12.04 kernel vulnerable?	No. Actual running package family and backports matter. This measured guest reproduced the flaw.
Is this remote exploitation?	The demonstrated prerequisite is local ordinary code execution.
What does MAP_PRIVATE promise?	Private modifications should not propagate into the mapped backing file.

Question	Concise answer
Is the control just the attack minus a thread?	No. The control uses a writable private mapping and memcpy; the exploit uses a read-only mapping and /proc/self/mem.
Why not write directly through the pointer?	A PROT_READ pointer normally faults on a direct write; the exploit uses the memory interface and kernel race.
Why pwrite?	It combines the upstream seek/write pair; its offset here is a virtual process address.
What does MADV_DONTNEED do?	Discards relevant mapping/page state so access can refault/repopulate, racing the write/COW handling.
Why repeat the workers?	The outcome depends on scheduling/interleaving; repetition creates opportunities but no guaranteed win time.
Why 0000 instead of 0?	Equal-width replacement preserves the in-place field and separators; one byte would leave old digits.
Why is charlie's GID 1002?	That is the actually allocated group in this VM; UID and GID need not match.
Does a hash change prove root?	No. Complete-byte validation and a fresh non-sudo login with numeric UID 0 are separate checks.
Why did whoami say root?	UID 0 resolved to the root account name; numeric identity supplies the authority.
Why did the first authentication fail?	Its cause was not established. The failure is preserved; the second non-sudo attempt succeeded.
Does 0 seconds mean instantaneous or one attempt?	Neither. It is an integer timer reading; no kernel attempt count or finer race runtime was measured.
Did restoration revoke the old root shell?	No. It was explicitly exited and checked absent before restoring; a new ordinary login was then tested.
Did you test a patched guest?	No; optional S12 is not performed, discussion only.
Is a failed 15-second live run proof of patching?	No. It is a finite observation; inspect setup/errors and actual package evidence.
Does Member 5's symlink defence fix this?	No. The mechanisms are at different layers; this historical flaw needs a kernel fix.
How does this relate to authentication?	Login trusts account-database integrity; a kernel write bypass can change the identity applied at login without cracking the password.

Final rehearsal reminder

Confirm the course allowance, practise the actual transitions and explanation, record your spoken time, and clean up afterward. Keep screenshots/video outside the VM. Every new attempt gets a fresh label and is described according to its own output.